

RV32I 5-Stage Pipeline: Data Hazard / Forwarding 정리

2026.06.26 구현 및 검증 내용

1. 구현 요약

이번 작업은 기존 5-stage pipeline CPU에 데이터 포워딩 경로와 load-use hazard stall 동작을 추가하고, code_6_26 폴더만으로도 전체 CPU 구성을 확인할 수 있게 필요한 모듈을 같이 정리한 것이다.

구분	구현 내용	확인 포인트
Forwarding Unit	EX/MEM, MEM/WB 단계의 rd와 ID/EX 단계의 rs1/rs2를 비교해서 ALU 입력 mux select를 생성하였다. EX/MEM 결과가 더 최신 값이므로 MEM/WB보다 뒤에서 덮어쓰도록 구성하였다.	rd가 x0인 경우는 forwarding 대상에서 제외하였다.
ALU 입력 선택	rs1, rs2는 먼저 forwarding mux를 통과하고, ALU 두 번째 입력은 i_alu_src에 따라 imm 또는 forwarded rs2 중에서 선택하도록 분리하였다.	store 명령에서는 주소 계산은 rs1+imm, 저장 데이터는 forwarded rs2가 되도록 처리하였다.
Hazard Detect	ID 단계의 rs1/rs2와 EX 단계의 load 목적지 rd를 비교해 load-use hazard를 감지한다.	감지 시 PCwrite=0, IF_ID_write=0, bubble=1로 한 사이클 stall을 넣는다.
CPU Core 정리	앞으로 ID stage에서 branch 판단과 branch forwarding을 추가하기 쉽도록 주요 wire 선언을 ID stage 쪽 forward declaration 블록으로 모았다.	PC/IF에서 바로 필요한 신호는 기존 위치에 유지하였다.

2. 파일 구성

오늘 제출용 폴더는 Risc_V/code_6_26이며, 6_24_25에서 그대로 사용하는 모듈도 같이 복사해 단독으로 컴파일할 수 있게 맞췄다.

파일	역할
cpu_core.v	5-stage pipeline top. hazard detect, forwarding unit, ALU, pipeline register 연결을 포함한다.
Forwarding_unit.v	EX/MEM 및 MEM/WB 단계의 결과를 ALU 입력으로 forwarding하기 위한 select 생성.
hazard_detect.v	load-use hazard 감지 후 PC/IF-ID 정지와 ID/EX bubble 삽입 제어.
alu.v	forwarded rs1/rs2, immediate 선택, branch 비교, store data 전달.
PC.v, IF_ID_reg.v, ID_EX_reg.v, reg_file.v	PC update, IF/ID stall 유지, ID/EX 제어/데이터 전달, register file bypass read.
Control.v, alu_controller.v, imm_gen.v, bta.v, EX_MEM_reg.v, MEM_WB_reg.v, dmem.v, imem.v	기존 6_24_25 구현을 재사용한 의존 모듈.
tb_load_data_hazard.v, program.hex	load-use stall 및 data forwarding 검증용 테스트벤치와 instruction memory image.

3. 검증 결과

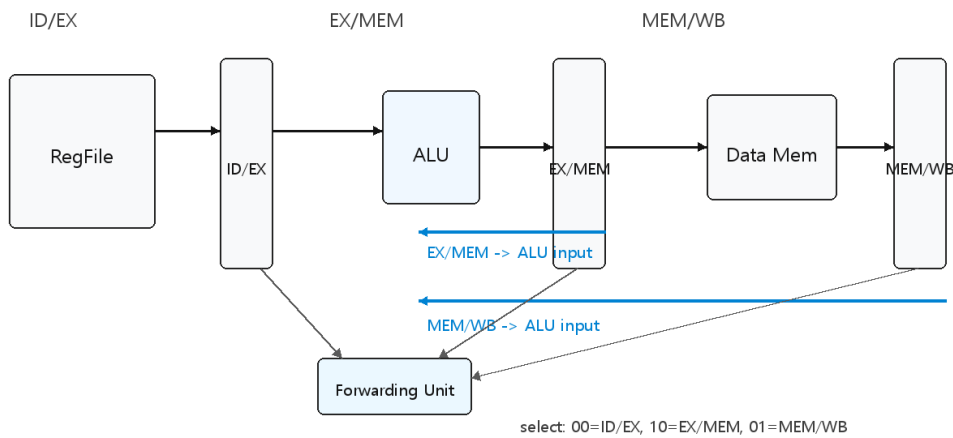
Questa에서 tb_load_data_hazard를 실행해 load-use stall과 forwarding 결과를 확인하였다. 테스트는 lw 직후 ALU 사용, ALU 결과 store data forwarding, lw 직후 store data 사용을 포함한다.

검증 항목	기대값	결과
lw x1, 0(x0) 후 add x2, x1, x1	x1=7, x2=14	PASS
add/addi 결과 forwarding	x3=19, x5=38, x7=45	PASS
store data forwarding	dmem[1]=19, dmem[2]=7	PASS
최종 테스트벤치 출력	PASS: load-use stall and data forwarding test	PASS

4. 참고 도식

아래 그림은 강의자료 캡처가 아니라, 구현 내용을 설명하기 위해 직접 단순화해서 다시 그린 도식이다.

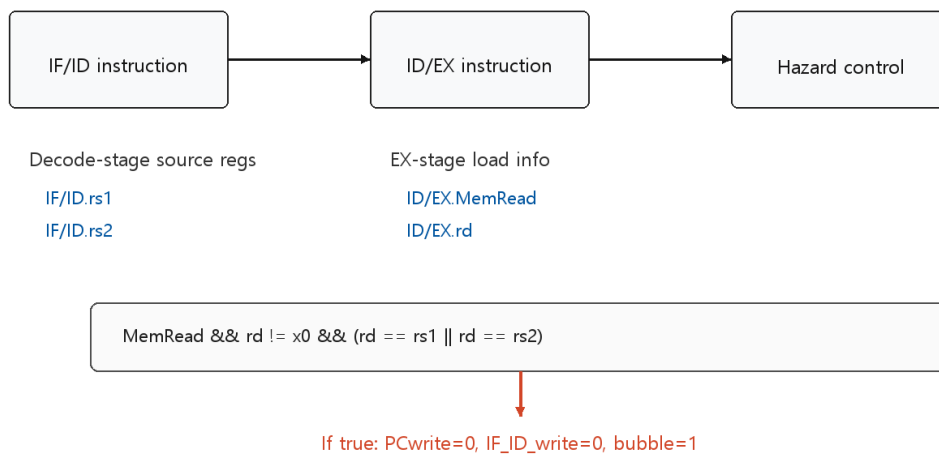
Forwarding path used in this CPU



Original simplified diagram based on the pipeline forwarding concept.

그림 1. Forwarding 경로 단순 도식

Load-use hazard detection used in this CPU



Original simplified diagram based on the load-use stall condition.

그림 2. Load-use hazard 감지 조건 단순 도식

5. 다음 구현 예정

항목	내용
ID-stage branch	branch 판단을 EX보다 앞선 ID stage에서 수행하도록 comparator와 branch target 계산 위치를 조정할 예정이다.
Branch operand forwarding	ID stage에서 branch 비교를 하면 rs1/rs2 값도 최신값이 필요하므로 EX/MEM/WB에서 ID stage로 들어오는 forwarding 경로가 필요하다.
Control hazard 처리	branch taken 시 IF/ID flush, 필요 시 ID/EX bubble 정책을 정리해야 한다.

참고 문헌

David A. Patterson, John L. Hennessy, Computer Organization and Design RISC-V Edition: The Hardware/Software Interface, 2nd Edition, Morgan Kaufmann.